

LAB 2

ECE 3829: Advanced Digital System
Design with FPGAs

2/17/2025

James Santacroce

Diagrams:

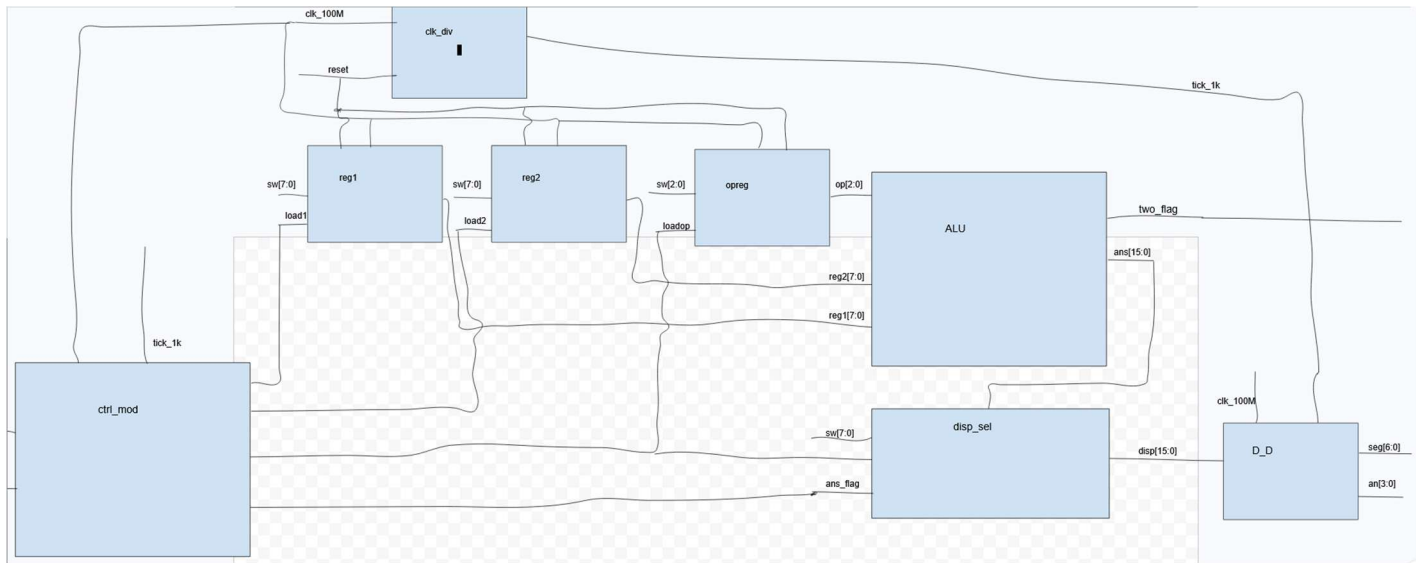


Figure 1: Block diagram of Lab 2.

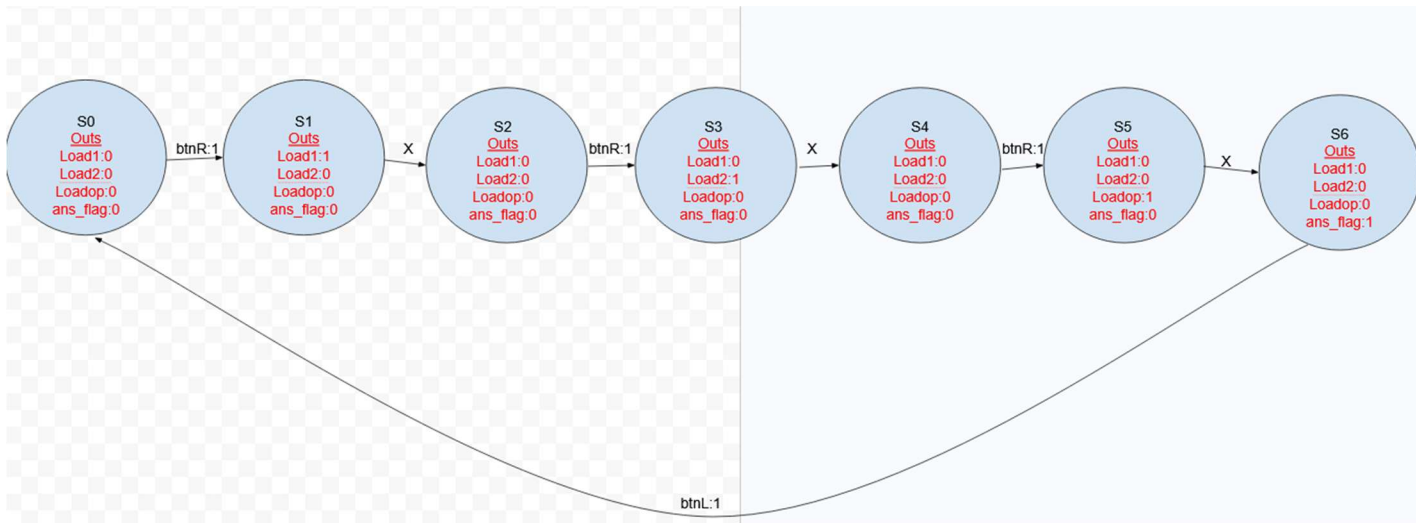


Figure 2: State diagram of control module.

For lab two, the block diagram in figure 1 was used to create a calculator. It features eight blocks (seven distinct blocks). There are two control input signals, reset and btnR, reset being the left button, btnR being the right button. There is only one data input signal, sw[7:0], used to control operands and to select which operation to use, though, for the operation, only switches 2 through 0 are used. Two_flag is a control output, as it just controls one LED and the hex mode being displayed (twos complement, or standard hex). The two data outputs are seg[6:0] and an[3:0], being used to control the seven-segment display. There are several control and data

signals between each block, such as `ans[15:0]`, `ans_flag`, `load1`, and `load2`. Two timing control signals are used, `clk_100M`, and `tick_1k`.

The state-machine (figure 2) has seven distinct states. States `S1`, `S3`, and `S5` are purely buffer states, used only to output control load signals. `S0`, `S2`, `S4`, and `S6` are wait states, which wait for user input to determine operands (`S1`, `S3`), operation (`S4`), or a restart signal (`S6`).

Operation:

The goal of this design is for the board to act as a calculator. Upon powering the FPGA, the user can use the DIP switches to choose an 8-bit number, which is then displayed on the 7-segment display in hexadecimal representation. Since the requirements weren't explicit in how this should be done, it was determined the easiest way to do this would simply be to display the live switch values as they were chosen. This makes it so the user can see exactly what number is being chosen without having to convert from binary to hexadecimal every time a number is chosen. This, however, also required a separate module `disp_sel`, so that the live feed or the answer could be selected to be displayed, based on the current state.

After selecting the first operand, the right button is pressed, and the operand is saved to an 8-bit register. The process is repeated for the second operand, and then `S3/S4` is entered. At this point, the user selects the operation to perform on the two operands using the three rightmost switches. `000` selects XOR, `001` selects AND, `010` selects OR, `011` selects ADD, and `100` selects SUBTRACT. The right button is pressed again, and an answer is given on the 7-segment display in hexadecimal format, and the FPGA waits for the left button (reset) to be pressed to restart, displaying the answer indefinitely. If subtract mode is chosen, the FPGA has the ability to enter twos complement mode. In this mode, an LED lights up to indicate to the user that the displayed answer is in its twos complement form and is thus a negative value.

The first module developed was the `clk_div` module, as it was simply taken from the lab one `clk_div` module and modified (1Hz clock signal was not necessary in this lab). It increments a counter `C2`, until `C2` is equal to `99999`. Once it is, an output signal `tick_1k` is set high, and this signal is then used the modules, as an enable signal. In the case of this specific

calculator, it is only used in the display driver, as it is the only module that requires a relatively low clock signal.

The module `ctrl_mod` functions as the brain of the calculator, containing the state machine designed for the lab. Also contained in `ctrl_mod` is logic to generate a pulse when the right button is pressed, preventing multiple consecutive button presses that cycle through the states. This was developed during lab one, as one error that was discovered in that lab was that the button signal bounced and cycled through the states. It stores a button press in two separate flip-flops, `btnR_sync` and `btnR_prev`. However, `btnR_sync` stores the direct input from `btnR`, and `btnR_prev` stores `btnR_sync`. These two values are then compared by ANDing `btnR_sync` and NOT `btnR_prev`. This way, if `btnR` is currently pressed (1) and `btnR_prev` indicates `btnR` was already pressed (1), you get 0 assigned to `btnR_pulse...` preventing multiple presses.

It was decided to separate each load state into their own, self-contained buffer states. This is due to another issue encountered in lab one, which required implementing a delay to a load signal to overcome. Rather than doing that again, it was determined the best way to avoid this from the beginning would be to implement these buffer states. It makes the `ctrl_mod` design bulkier, with more states, but ensures that two values won't ever be accidentally loaded at the same time, ensuring smooth operation.

`Reg1` is a register module, used to store both operands (it is instantiated twice in the top layer module). It operates off the 100MHz clock, and outputs an 8-bit number. `Reg_op` is a similar register, except it stores the 3 bits used to determine operation. `Alu` is the arithmetic logic unit, this unit is responsible for all of the math done by the calculator. It received both operands as 8 bit numbers, and the operation as a 3 bit number, and outputs a 16-bit answer (`ans[15:0]`), and a two's complement flag (`two_flag`). The `op[2:0]` is used to select which math to do in a switch/case block. A 16-bit output was chosen due to the ease of segmenting the output in the 7-segment display (0000 0000.... each 4 digits control one digit on the display).

`Disp_sel` has only one job, choose whether to output the live switch feed, or the answer. The answer comes from the ALU, and the switch feed comes directly from the switches. The control input `ans_flag` comes from the control module itself, specifically from `S6`. If the live feed is being displayed, 8 bits are appended to the beginning of `sw`, and 13 bits are appended to the front of `sw` if operation is being displayed. `D_D`

is the display driver for the calculator. The D_D module from lab one was used and modified to include hex digits A through F, representing 10 through 15.

To verify the operation of the control module, a test bench was developed. Reset is initialized to one at the start and is turned off after three clock pulses. It then pulses btnR to load the first register, waits, pulses it again to load the second, waits, pulses it again to load the operation, and outputs the answer flag signal. To be thorough, a reset signal is sent again to ensure return to S0. The waveform is shown in figure 3, and includes the state signal, so that the transitions can be monitored exactly. Each buffer state (1, 3, 5) only lasts a clock cycle as desired and indicates desired behavior.



Figure 3: ctrl_mod simulation.

Conclusion:

The design in total uses about 8 microwatts of power on a typical process, and the total implemented design is shown below in figure 4. The device functions completely as required, and there are no major fixes necessary. There are a few quality-of-life suggestions to enhance usability, however. One such improvement would be to include an indicator to show that a register has been loaded. Since the life feed from the switches is used to display what operand/operation is being chosen, the user has no way of knowing if their choice has been selected, so it is possible to "mis-click" and mess up the operation. The live-feed could and should be kept to enhance usability, but an LED indicator could be implemented so the user knows when it is time to enter the next value. It also may be helpful to display all zeroes and implement a two second wait period when moving from operand to operand to operation, to make it more obvious.

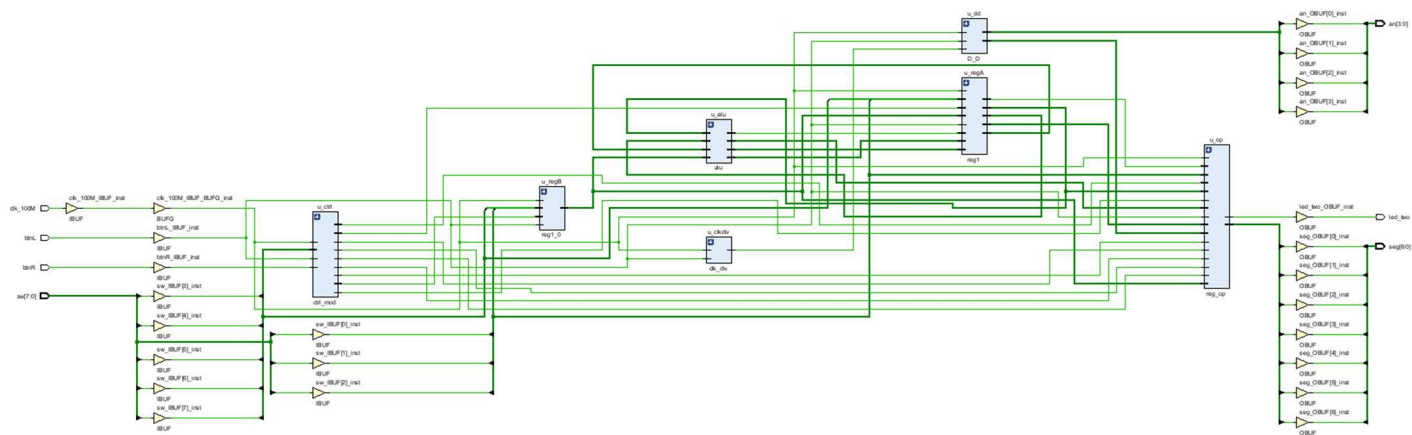


Figure 4: Final implemented design.